INF1037
Programação em C

2026.1 - Turma 33C
Prof. Adriano Branco
Aula 24: 27/05/2026

Slides da aula da Profa. Paula Rodrigues

Pontifícia Universidade Católica do Rio de Janeiro
Departamento de Informática

Conteúdo Programático

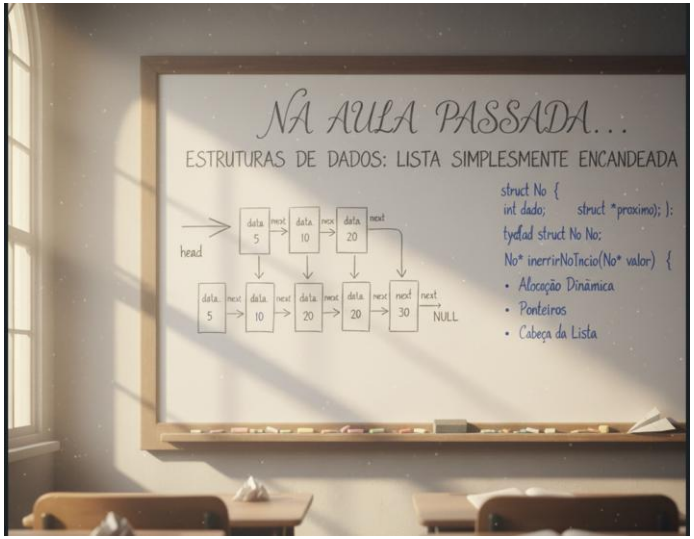
- Tipos Estruturados ✅
- Vetor de Tipos Estruturados ✅
- União e Enumeração ✅
- Arquivo Binário ✅
- Lista
- Recursão
- Segunda Prova (P2)

DEPARTAMENTO DE INFORMÁTICA
PUC·RIO

Listas Simplesmente Encadeadas
(Parte 2)

Pontifícia Universidade Católica do Rio de Janeiro
Departamento de Informática

Relembrando...



- Definição de uma lista:

```
/*Defininição da lista*/
typedef struct no_lista NoLista;
struct no_lista {
  conteudo dado;
  NoLista* proximo;
};
```

Funções desenvolvidas:

- Criar lista:

```
NoLista* cria_lista();
```

- Inserir elemento no início da lista:

```
void insere_inicio_lista(NoLista** no_lista, conteudo valor);
```

- Imprimir os elementos da lista:

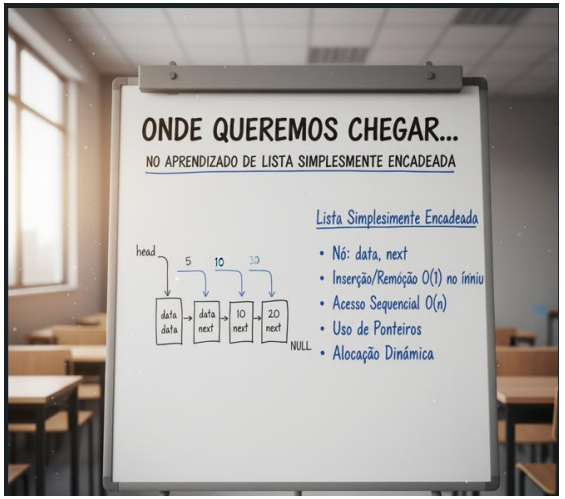
```
void imprime_lista(const NoLista* lista);
```

- Verificar se um elemento pertence à lista:

```
int pertence_lista(NoLista* lista, conteudo elem);
```

Relembrando...

- Nossos próximos passos:
  - Inserir no final
  - Inserir de forma ordenada
  - Remover um elemento
  - Liberar área de memória alocada para um nó e para a lista
  - Buscar elemento por um campo
  - Salvar e carregar a lista em um arquivo binário
  - Ordenar a lista por um determinado critério



Função para inserir um elemento no final da lista

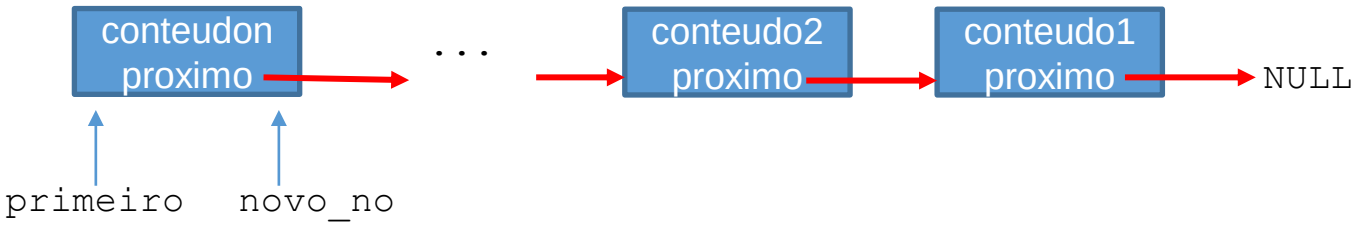
- A função de inserção apresentada na “Parte 1” armazena os elementos na lista na ordem inversa à ordem de inserção, pois um novo elemento é sempre inserido no início da lista.

- Qual a ideia se quisermos inserir um novo nó no final da lista?



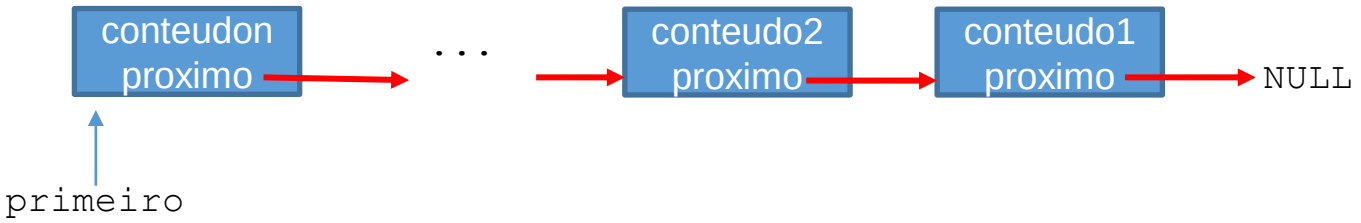
Função para inserir um elemento no final da lista

- Pensando que já inserirmos *n* elementos na lista, ela estaria assim:

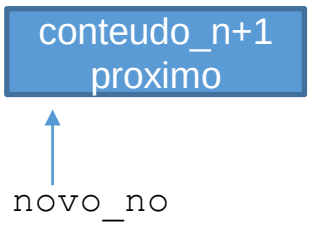


Função para inserir um elemento no final da lista

- Pensando que já inserirmos *n* elementos na lista, ela estaria assim:



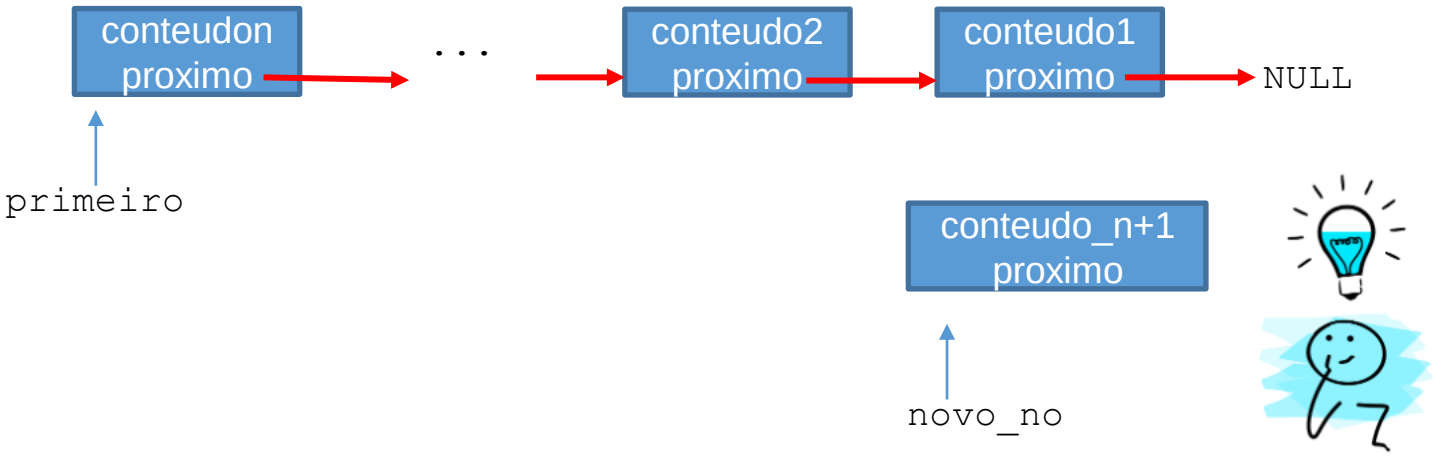
- Vamos criar um novo nó... E vamos andar até onde para colocarmos ele no final da lista?





Função para inserir um elemento no final da lista

- Vamos andar até o `no_atual.próximo == NULL` e vamos inserir ele lá!



Função para inserir um elemento no final da lista



```c
void insere_final_lista(NoLista** primeiro, int valor) {
  NoLista* novo_no = (NoLista*)malloc(sizeof(NoLista));
  if (novo_no == NULL) {
    printf("Nao foi possivel alocar memoria para um novo no na lista\n");
    exit(1);
  }
  /* uma vez que o no foi criado, estamos inicializando seus campos */
  novo_no->dado = valor;
  novo_no->proximo = NULL; /* este vai passar a ser o ultimo nó, entao seu
                              proximo vai apontar para NULL */

  /* se a lista ainda nao tem nenhum elemento */
  if (*primeiro == NULL) {
    *primeiro = novo_no;
  }
  else {
    /* andando até o final da lista */
    NoLista* no_atual = *primeiro;
    while (no_atual->proximo != NULL) {
      no_atual = no_atual->proximo;
    }
    no_atual->proximo = novo_no;
  }
}
```

Função para inserir um elemento no final da lista





```
int main(void) {
  NoLista* primeiro_no = NULL;
  printf("Forneca quantos valores deseja inserir na lista:  ");
  int qtd;
  scanf(" %d", &qtd);
  printf("Inserindo no inicio... \n");
  int i;
  for (i = 0; i < qtd/2; i++) {
    insere_inicio_lista(&primeiro_no, i+1);
  }
  printf("Imprimindo depois de inserir no inicio: \n");
  imprime_lista(primeiro_no);
  printf("Inserindo no final... \n");
  for (int j = i; j < qtd; j++) {
    insere_final_lista(&primeiro_no, j + 1);
  }
  printf("Imprimindo depois de inserir no final: \n");
  imprime_lista(primeiro_no);
  ...
  return 0;
}
```

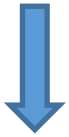
```
Forneca quantos valores deseja inserir na lista:  6
Inserindo no inicio...
Imprimindo depois de inserir no inicio:
        Valor do no: 3
        Valor do no: 2
        Valor do no: 1
Inserindo no final...
Imprimindo depois de inserir no final:
        Valor do no: 3
        Valor do no: 2
        Valor do no: 1
        Valor do no: 4
        Valor do no: 5
        Valor do no: 6
```

Função para inserir e manter a lista ordenada

- Se quisermos manter os elementos na lista em uma determinada ordem, temos que encontrar a posição correta para inserir o novo elemento.

Função para inserir e manter a lista ordenada

- Se quisermos manter os elementos na lista em uma determinada ordem, temos que encontrar a posição correta para inserir o novo elemento.



Essa função não é eficiente, pois temos que percorrer a lista, elemento por elemento, para achar a posição de inserção.

Função para inserir e manter a lista ordenada

- Se quisermos manter os elementos na lista em uma determinada ordem, temos que encontrar a posição correta para inserir o novo elemento.

Essa função não é eficiente, pois temos que percorrer a lista, elemento por elemento, para achar a posição de inserção.

Se a ordem de armazenamento dos elementos dentro da lista não for relevante, optamos por fazer inserções no início, pois o custo computacional desta operação independe do número de elementos da lista.

Função para inserir e manter a lista ordenada

- Se quisermos manter os elementos na lista em uma determinada ordem, temos que encontrar a posição correta para inserir o novo elemento.

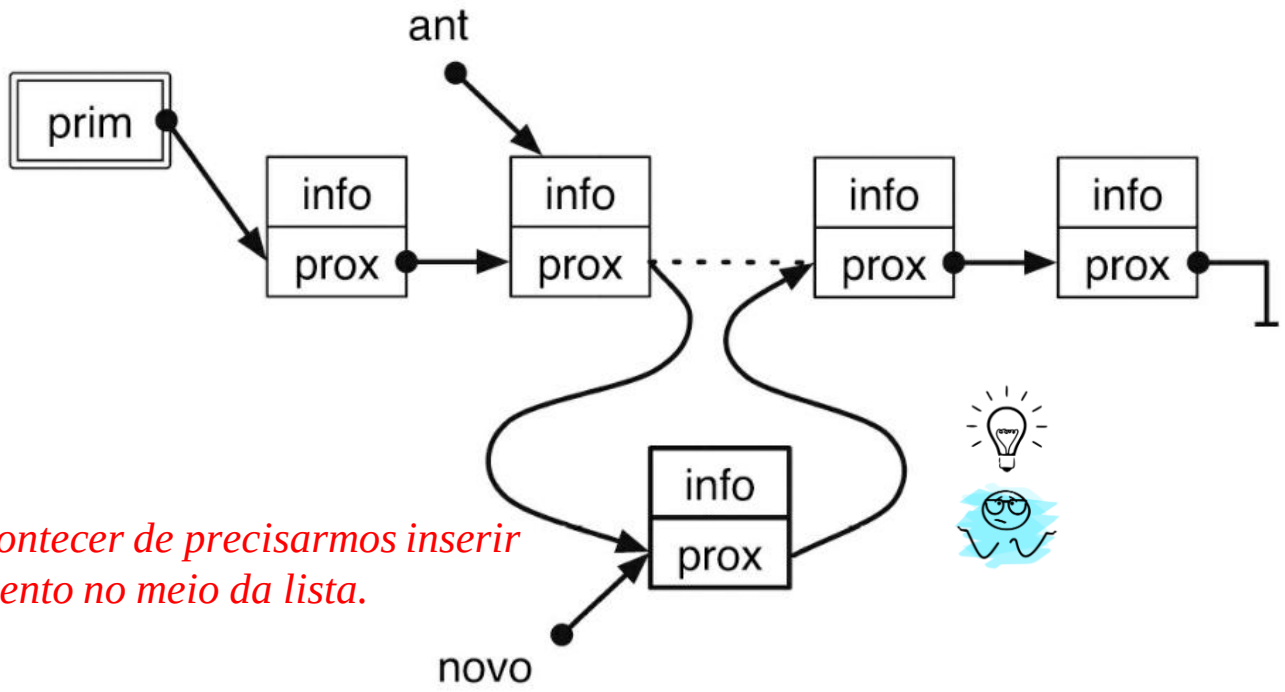
Essa função não é eficiente, pois temos que percorrer a lista, elemento por elemento, para achar a posição de inserção.

Se a ordem de armazenamento dos elementos dentro da lista não for relevante, optamos por fazer inserções no início, pois o custo computacional desta operação independe do número de elementos da lista.

No entanto, se desejarmos manter os elementos em ordem, cada novo elemento deve ser inserido na ordem correta.

Função para inserir e manter a lista ordenada

- Exemplo: considerar que queremos manter nossa lista de números inteiros em ordem crescente.



Pode acontecer de precisarmos inserir um elemento no meio da lista.

Função para inserir e manter a lista ordenada



```
Função que insere um elemento na lista de forma ordenada:
1. devemos achar o elemento da lista que irá preceder o novo elemento
(elemento a ser inserido).
2. com o ponteiro para o elemento que antecede o novo elemento,
o novo elemento vai apontar para o próximo elemento e o elemento precedente
apontará para o novo.
3. se o elemento tiver que ser inserido antes do primeiro elemento,
recaímos no caso de inserção no início da lista.*/
void insere_ordenada_lista(NoLista** primeiro, int valor) {
  /* ponteiro para o elemento anterior */
  NoLista* no_anterior = NULL;
  /* ponteiro para percorrer a lista */
  NoLista* no_atual = *primeiro;
  /* localiza a posicao para insercao do novo conteudo */
  while ((no_atual != NULL) && (no_atual->dado < valor)) {
    no_anterior = no_atual;
    no_atual = no_atual->proximo;
  }
  ...
```

Função para inserir e manter a lista ordenada





```
void insere_ordenada_lista(NoLista** primeiro, int valor) {
  ...
  /* cria o novo no */
  Lista* novo_no = (NoLista*)malloc(sizeof(NoLista));
  if (novo_no == NULL) {
    printf("Nao foi possivel alocar memoria para um novo no na lista\n");
    exit(1);
  }
  novo_no->dado = valor;
  /* encadeia o novo elemento */
  /* insercao no inicio */
  if (no_anterior == NULL) {
    novo_no->proximo = *primeiro;
    *primeiro = novo_no;
  }
  /* insere no meio da lista */
  else {
    novo_no->proximo = no_anterior->proximo;
    no_anterior->proximo = novo_no;
  }
}
```

Função para inserir e manter a lista ordenada



```c
int main(void) {
  NoLista* primeiro_no = NULL;
  printf("----\n");
  printf("insere 23.\n");
  insere_ordenada_lista(&primeiro_no, 23);
  printf("saida:\n");
  imprime_lista(primeiro_no);
  printf("----\n");
  printf("insere 45.\n");
  insere_ordenada_lista(&primeiro_no, 45);
  printf("saida:\n");
  imprime_lista(primeiro_no);
  printf("----\n");
  printf("insere ordenado 10.");
  insere_ordenada_lista(&primeiro_no, 10);
  printf("saida:\n");
  imprime_lista(primeiro_no);
  printf("----\n");
  printf("insere ordenado 30.");
  insere_ordenada_lista(&primeiro_no, 30);
  printf("saida:\n");
  imprime_lista(primeiro_no);
  printf("----\n");
  printf("insere ordenado 50.\n");
  insere_ordenada_lista(&primeiro_no, 50);
  printf("saida:\n");
  imprime_lista(primeiro_no);
  return 0;
}
```

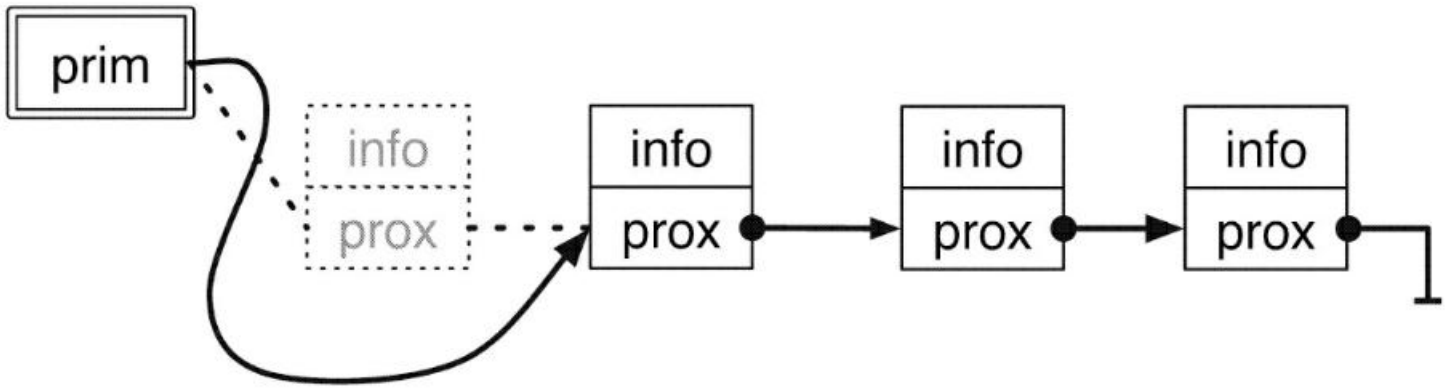
```
----
insere 23.
saida:
        Valor do no: 23
----
insere 45.
saida:
        Valor do no: 23
        Valor do no: 45
----
insere ordenado 10.saida:
        Valor do no: 10
        Valor do no: 23
        Valor do no: 45
----
insere ordenado 30.saida:
        Valor do no: 10
        Valor do no: 23
        Valor do no: 30
        Valor do no: 45
----
insere ordenado 50.
saida:
        Valor do no: 10
        Valor do no: 23
        Valor do no: 30
        Valor do no: 45
        Valor do no: 50
```

Função que retira um elemento da lista

- Precisamos achar onde está o elemento a ser retirado na lista:



Se descobrirmos que o elemento a ser retirado é o **primeiro** da lista, devemos fazer com que o valor do primeiro elemento da lista passe a ser o ponteiro para o segundo elemento da lista, e então podemos liberar o espaço alocado para o elemento que queremos retirar.

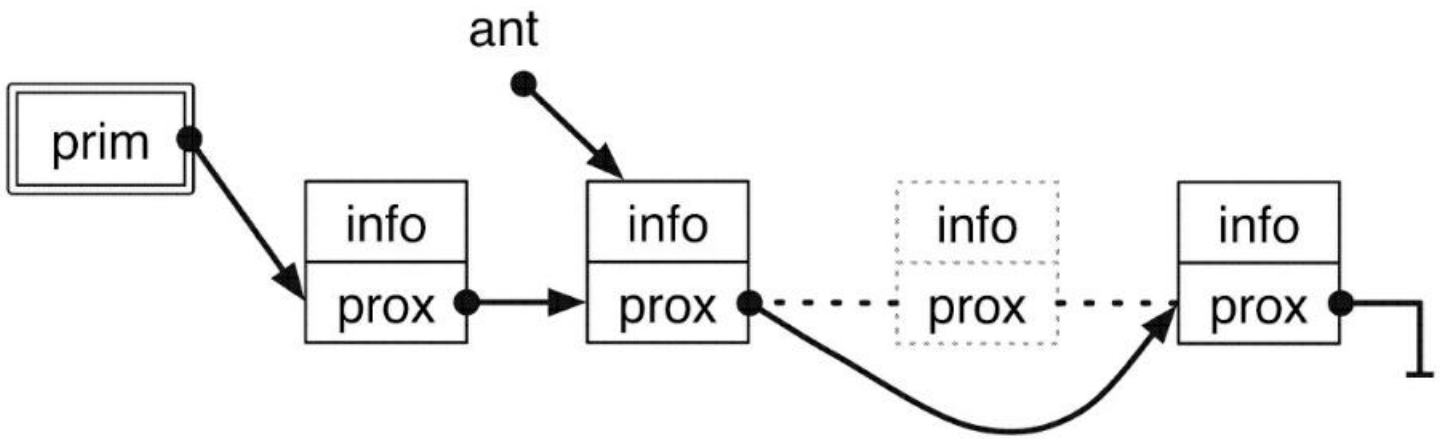


Função que retira um elemento da lista

- Precisamos achar onde está o elemento a ser retirado na lista:



Se o elemento a ser removido estiver no **meio** da lista, devemos fazer com que elemento anterior a ele passe a apontar para o elemento seguinte, e então podemos liberar o elemento que queremos retirar. Portanto, aqui também precisamos do ponteiro para o elemento anterior para poder acertar o encadeamento da lista.



Função que retira um elemento da lista



```c
void remove_elemento_lista(NoLista** primeiro_no, int elem) {
  NoLista* no_anterior = NULL;
  NoLista* no_atual = *primeiro_no;
  /* procura o elemento da lista que se deseja remover, guardando
     no "no_anterior" */
  while ((no_atual != NULL) && (no_atual->dado != elem)) {
    no_anterior = no_atual;
    no_atual = no_atual->proximo;
  }
  /* verifica se achou o elemento */
  if (no_atual != NULL) {
    /* retira o elemento do inicio */
    if (no_anterior == NULL) {
      *primeiro_no = no_atual->proximo;
    }
    /* retira o elemento do meio da lista */
    else {
      no_anterior->proximo = no_atual->proximo;
    }
    /* libere o no atual */
    free(no_atual);
  }
}
```

Função que retira um elemento da lista



```c
int main(void) {
  NoLista* primeiro_no = NULL;
  printf("insere 23.\n");
  insere_ordenada_lista(&primeiro_no, 23);
  printf("insere 45.\n");
  insere_ordenada_lista(&primeiro_no, 45);
  printf("insere ordenado 10.\n");
  insere_ordenada_lista(&primeiro_no, 10);
  printf("insere ordenado 30.\n");
  insere_ordenada_lista(&primeiro_no, 30);
  printf("insere ordenado 50.\n");
  insere_ordenada_lista(&primeiro_no, 50);
  printf("\nELEMENTOS DA LISTA:\n");
  imprime_lista(primeiro_no);
  printf("removendo o elemento 10:\n");
  remove_elemento_lista(&primeiro_no, 10);
  imprime_lista(primeiro_no);
  printf("removendo o elemento 45:\n");
  remove_elemento_lista(&primeiro_no, 45);
  imprime_lista(primeiro_no);
  printf("removendo o elemento 50:\n");
  remove_elemento_lista(&primeiro_no, 50);
  imprime_lista(primeiro_no);
  return 0;
}
```

```
insere 23.
insere 45.
insere ordenado 10.
insere ordenado 30.
insere ordenado 50.

ELEMENTOS DA LISTA:
        Valor do no: 10
        Valor do no: 23
        Valor do no: 30
        Valor do no: 45
        Valor do no: 50
removendo o elemento 10:
        Valor do no: 23
        Valor do no: 30
        Valor do no: 45
        Valor do no: 50
removendo o elemento 45:
        Valor do no: 23
        Valor do no: 30
        Valor do no: 50
removendo o elemento 50:
        Valor do no: 23
        Valor do no: 30
```

Função para verificar se a lista está vazia e a função para liberar



```c
/* retorna 1 se a lista estiver vazia e 0 caso contrário */
int lista_vazia(const NoLista* primeiro_no) {
  return (primeiro_no == NULL);
}
```

Função para verificar se a lista está vazia e a função para liberar



```c
int main(void) {
  NoLista* primeiro_no = NULL;
  if (lista_vazia(primeiro_no)) {
    printf("A lista esta vazia.\n");
  }
  else {
    printf("A lista nao esta vazia.\n");
  }
  insere_ordenada_lista(&primeiro_no, 23);
  insere_ordenada_lista(&primeiro_no, 45);
  insere_ordenada_lista(&primeiro_no, 10);
  insere_ordenada_lista(&primeiro_no, 30);
  insere_ordenada_lista(&primeiro_no, 50);
  remove_elemento_lista(&primeiro_no, 10);
  remove_elemento_lista(&primeiro_no, 45);
  remove_elemento_lista(&primeiro_no, 50);
  printf("\nELEMENTOS DA LISTA:\n");
  imprime_lista(primeiro_no);
  if (lista_vazia(primeiro_no)) {
    printf("A lista esta vazia.\n");
  }
  else {
    printf("A lista nao esta vazia.\n");
  }
  return 0;
}
```



```
A lista esta vazia.
insere 23.
insere 45.
insere ordenado 10.
insere ordenado 30.
insere ordenado 50.
removendo o elemento 10:
removendo o elemento 45:
removendo o elemento 50:

ELEMENTOS DA LISTA:
        Valor do no: 23
        Valor do no: 30
A lista nao esta vazia.
```

Função de destrói a lista



```c
/* A função libera o encadeamento dos elementos e depois libera a estrutura o
ponteiro que aponta para o inicio da lista. Para liberar o encadeamento, a
função percorre elemento por elemento, liberando-os. É importante observar que
devemos guardar a referência para o próximo elemento antes de liberar o
elemento corrente.*/
void destroi_lista(NoLista** primeiro_no) {
  NoLista* no_atual = *primeiro_no;
  while (no_atual != NULL) {
    NoLista* proximo_no = no_atual->proximo;
    free(no_atual);
    no_atual = proximo_no;
  }
  free(*primeiro_no);
}
```

Prática

Estamos gerando a lista de alunos matriculados no curso de INF1037. Para tal criamos a seguinte estrutura:

```
typedef struct no_lista_nomes NoListaNomes;
struct no_lista_nomes {
  char conteudo[100];
  NoListaNomes *proximo;
};
```

Crie as seguintes funções para manipular sua lista com os nomes dos alunos:

- `NoListaNomes* cria_no(const char *texto)` - cria um nó com cópia da string.
- `void insere_inicio(NoListaNomes **lista, const char *texto)` – insere um nó no início da lista.
- `void insere_fim(NoListaNomes **lista, const char *texto)` – insere um nó no final da lista.
- `void imprime(NoListaNomes **lista)` – imprime todos os elementos da lista, um por linha.
- `int remove_no (NoListaNomes **lista, const char *texto)` – remove o primeiro nó que contenha a string `texto`. Retorna 1 se remover ou 0 caso não exista;
- `int conta_nomes (NoListaNomes **lista, const char *texto)` – conta o número de ocorrências de `texto` na lista. Retorna 0 caso `texto` não seja um elemento da lista.
- `void libera_lista(NoListaNomes **lista)` – libera toda a memória alocada.